Magic: The Gathering Multiplayer Network Protocol

(MTGNP) — Machine Problem Rubric & Instructions

CSNETWK • T3 AY 2025–2026

Objective

This machine problem requires you to implement a networked two-player card game system following the **Magic: The Gathering Multiplayer Network Protocol (MTGNP) v1.0**, as defined in RFC 0001 (CSNETWK). The primary learning goals are:

Magic: The Gathering is one of the most rules-dense games ever designed. This is intentional: the protocol reflects exactly the kind of layered, stateful, and specification-heavy system you will encounter when working with real-world network protocols. Successfully implementing MTGNP is not about memorising card game rules; it is about demonstrating that you can **read a formal specification and build something that faithfully follows it**. The same discipline applies to any protocol you implement in industry.

- Applying TCP socket programming to a real, non-trivial protocol specification.
- Reading and correctly implementing a formal RFC (including PDU structure, message framing, sequence number handling, and error codes)
- Building a client-server architecture where the server is the sole authoritative source of game state.
- Handling concurrent connections, heartbeat / keep-alive mechanics, and graceful disconnect/reconnect.

Your implementation will be evaluated both on protocol correctness and on your ability to explain the socket-level behaviour of your code during the demo.

Deliverables

- Source code (server + client) in a single project repository or ZIP archive
  - I suggest using git for version control and keeping track of individual member's contributions but this is not required.
- README file in PDF containing:
  - Build and run instructions, including how to enable verbose mode
  - Work Distribution Matrix (see Groupings section)
  - AI Usage section — all tools used and how they were used
  - Any known limitations or deviations from the RFC

Demo

Deadline and Demo Date: 13th and 14th week (exact date to be announced by your instructor). All group members must be present for the demo.

⚠️ **Verbose Mode Prerequisite**

Your implementation must support a verbose mode that can be toggled on and off at runtime (e.g., via a command-line flag or startup argument). When enabled, verbose mode must print all PDUs sent and received, both client-side and server-side, to the console in a readable, clearly labelled format. **The MP will not be checked unless verbose mode is working and active during the demo. The MP will be an automatic zero.** Ensure both your client and server can be started in verbose mode before your checking slot.

⚠️ **Grading Deductions**

- If any question relating to the submitted work, including implementation decisions, protocol behaviour, socket programming, or AI-generated content, is not answered properly during the demo, deductions in grades may be given for up to a grade of zero.
- The instructor may adjust individual grades based on group contribution.

AI Usage Policy

Students **may** use AI tools (e.g., ChatGPT, GitHub Copilot, Claude) to assist with this machine problem. However, the following rules apply:

- You must document every AI tool used and describe how it was used in the README's AI Usage section.
- All AI-generated code must be reviewed, tested, and verified by the student before submission.
- The final submission must reflect your own understanding. Every member must be able to explain any part of the code — including AI-generated sections — during the demo.
- Blindly copying AI output without testing or comprehension is not allowed. Submissions that show a lack of understanding or appear to be largely untested AI output may receive reduced credit, up to a grade of zero.
- Reusing AI-generated content from another student's session or sharing AI outputs between groups is prohibited.

The goal of this policy is to let you use AI as a **learning aid**, not a shortcut. Understanding TCP sockets and protocol implementation is the core skill being assessed.

Groupings

Each group should have up to four members. All members are expected to contribute meaningfully to the design, implementation, and testing of the project. Each participant must be capable of explaining all parts of the submission, even components they did not directly code.

A detailed report of tasks implemented by each member must be recorded in the README using the Work Distribution Matrix format below. Additional rows may be added as needed. **False reporting of work distribution is treated as academic misconduct.**

Task / Feature	Member 1	Member 2	Member 3	Member 4
TCP Server: connection handling, framing, dispatch				
Game lifecycle: LOBBY, GAME_SETUP, MULLIGAN logic				
Turn & phase engine (all phases/steps, transitions)				
Priority & Stack logic, spell/ability resolution				
Combat system (attackers, blockers, damage)				
Client implementation & state rendering				

Task / Feature	Member 1	Member 2	Member 3	Member 4
PDU serialisation/deserialisation (all 25 PDU types)				
Error handling, PING/PONG heartbeat, disconnect logic				
Verbose mode (client + server PDU logging, toggle on/off)				
Testing & interoperability				
README / documentation / AI disclosure				

Violations of academic integrity, such as unauthorized sharing of code, plagiarism, or misrepresenting AI-generated work, may result in a grade of zero and may be referred to the appropriate academic disciplinary body. Code sharing between different groups is strictly prohibited. While discussion of ideas, clarification, and interoperability testing is encouraged, all source code must be the original work of the group submitting it. Any third-party libraries or tools must be cited appropriately.

Non-cooperating group members

If a member consistently fails to contribute

1. The group must first attempt to resolve the issue through clear, documented internal communication.
2. If unresolved, notify the course instructor with supporting evidence (e.g., chat logs, Git commits).
    a. The group can decide to exclude non-contributing members.
    b. Or the instructor may adjust individual grades.

What Is and Is Not Allowed

The table below summarises academic integrity and tool-use policies for this machine problem.

Activity	✅ Allowed	❌ Not Allowed
Working with a group (2–4 people)	Yes – group work is supported	Groups larger than 4
Using AI tools (e.g., ChatGPT, Copilot, Claude)	Yes – to understand TCP/socket concepts, generate boilerplate, or debug code	Submitting AI-generated code without review, testing, or acknowledgment
Discussing protocol ideas with classmates	Yes – general discussion of MTGNP concepts is encouraged	Sharing source code or implementation files between groups
Testing with another group's client/server	Yes – for interoperability testing and debugging	Submitting shared or merged code from different groups
Using open-source libraries/utilities	Yes – with proper citation in README	Using uncredited third-party code
Submitting identical code as another group	No	Plagiarism, even with minor changes

Activity	✅ Allowed	❌ Not Allowed
Individual understanding of group submission	Each member must be able to explain the entire solution	Relying solely on one member or AI without full team involvement
Acknowledging AI usage	Must include an AI Usage section in README describing all tools used and how	Omitting any mention of AI assistance
AI-generated code submitted as-is	Only if reviewed, tested, and clearly acknowledged	Blindly copying AI output without comprehension or testing

Grading Rubric (Total: 120/100 points)

Base points: 100; bonus: 20. Verbose mode must be working before any criterion is evaluated — see prerequisite row below. Even with a fully functioning code, the points are subject to deductions if the student cannot explain their work and answer questions.

Category	Criterion	Points	Description
⚠️ **PREREQUISITE — Verbose Mode (checking will not proceed without this):** Both the client and server must support a verbose mode that can be toggled on and off (e.g., via a command-line flag or startup argument). When enabled, all PDUs sent and received — on both the client side and the server side — must be printed to the console in a readable, clearly labelled format. **The MP will not be checked unless verbose mode is working and active during the demo.**			
TCP Sockets & Networking	**TCP Server Setup & Client Accept**	10	Correct TCP server socket creation on port 4444, binding, listening, and accepting exactly two client connections. Additional connections must be refused. Server must handle client disconnect/reconnect within the implementation-defined timeout.
	Message Framing	5	All PDUs are correctly framed with a 4-byte big-endian length prefix followed by a valid UTF-8 JSON payload. Receiver reads exactly the indicated byte count before attempting to parse. PDUs must not exceed 65,535 bytes.
	PDU Structure & seq_num	5	Every PDU includes the required 'type' and 'seq_num' fields. Priority-bearing client PDUs echo the correct seq_num from the latest PRIORITY_GRANT or server request PDU. Server rejects stale seq_nums with ERROR code STALE_ACTION.
Game Lifecycle	**LOBBY & PLAYER_READY Handling**	10	Server correctly enters LOBBY on startup and after each GAME_OVER. PLAYER_READY PDUs are validated: non-empty player_id, unique IDs, deck list of 1–50 cards from the fixed set. Server rejects duplicates with DUPLICATE_ID and invalid decks with ILLEGAL_DECK.
	GAME_SETUP & MULLIGAN	5	Server initialises life totals at 20, shuffles decks, deals seven cards, and determines first player via random coin flip. London Mulligan rule: players who mulligan redraw a full hand and place N cards on the bottom when keeping after N mulligans.
	IN_GAME Phase & Step Transitions	10	Server broadcasts PHASE_TRANSITION messages for all turn phases and steps (Untap, Upkeep, Draw, Main 1, Combat, Main

Category	Criterion	Points	Description
			2, End, Cleanup). Phase-specific rules are enforced — e.g., land drops only in Main Phase, no non-mana spells during Untap.
	GAME_OVER & Session Restart	5	Server correctly detects win/loss conditions (life total ≤ 0, empty library on draw, CONCEDE) and broadcasts GAME_OVER with the appropriate reason. Returns to LOBBY state and awaits fresh PLAYER_READY PDUs on the same TCP connections.
Server-Side Game Logic	**Game State Management & Hidden Info**	10	Server maintains the single authoritative Game State. GAME_STATE_UPDATE messages are personalised per player — each player's hand is hidden from the opponent. Library counts, battlefield, graveyard, and stack contents are correctly reflected.
	Priority & Stack Resolution	10	Server correctly issues PRIORITY_GRANT to the appropriate player at each priority window. The stack (LIFO) is maintained correctly — spells and abilities push and resolve in order. Stack resolves when both players pass priority consecutively. STACK_PUSH and STACK_RESOLVE are broadcast. **At least 5 card effects (any) should be implemented to achieve the full point**.
	Combat System	10	Server correctly manages the full combat sequence: Declare Attackers, Declare Blockers, Assign Damage Order, optional First Strike Damage, and Combat Damage. Summoning sickness is enforced. COMBAT_DAMAGE_RESULT is broadcast with correct damage values.
Client Implementation	**Client Sending & State Rendering**	5	Client correctly sends all required PDU types (PLAYER_READY, MULLIGAN_CHOICE, CAST_SPELL, PLAY_LAND, PRIORITY_PASS, CONCEDE, etc.). Client accepts GAME_STATE_UPDATE as authoritative and renders visible state accurately, discarding any locally computed state.
	PING/PONG Heartbeat	5	Client sends PING PDUs at regular intervals (recommended: every 30 s) and disconnects if no PONG is received within the implementation-defined timeout (recommended: 10 s). Server responds to every PING with a matching PONG.
Error Handling & Code Quality	**Error PDU Handling**	5	Server sends ERROR PDUs with correct error codes (STALE_ACTION, ILLEGAL_ACTION, ILLEGAL_DECK, DUPLICATE_ID, etc.) in all cases specified by the RFC. Client handles ERROR PDUs gracefully without crashing.
	Readability & Comments	5	Code is well-structured with meaningful variable and function names, and includes clear comments explaining non-obvious logic — especially socket setup, message framing, and protocol state transitions.
Bonus	**Full Card Effects**	10	Implementation of all Card Abilities and Effects.
	Bonus Features	10	Additional features beyond the base specification: triggered-ability UI, spectator client, graphical interface, or other creative extensions. Must be demonstrated and explained during checking.
TOTAL		**100 points + 10 bonus**	

Instructor AI Disclaimer

The MTGNP RFC (RFC 0001, CSNETWK) is the primary work of the course instructor. AI tools — primarily Claude — was leveraged to assist with RFC formatting and JSON schema examples. All AI-generated content was thoroughly reviewed, validated, and adapted to meet the functional and educational goals of this course. The protocol design, game logic specification, and pedagogical structure reflect the instructor's original intent.